

NULL-Werte der Datenbank



The screenshot shows the website **arbeitsagentur.de** with the header "Ein Service der Bundesagentur für Arbeit". The main content area is titled "SCHNELLÜBERSICHTEN | LABOUR" and includes a "Zurück" link. Below this, there is a section for "Bundesrepublik Deutschland" which contains a table with the following data:

Berichtsmonat	Januar 2005
Region	Bundesrepublik Deutschland
Arbeitslose	NULL
± zum Vorjahresmonat	NULL
Arbeitslosenquote	NULL
Vorjahresmonat	NULL
Gemeldete Stellen	NULL
± zum Vorjahresmonat	NULL

DBM-0001-2012-01875

NULL-Werte – Bedeutung



- NULL-Werte sind entgegen ihrem Namen eben keine Werte
- NULL-Werte zeigen an, dass ein Wert fehlt, zum Beispiel weil zur Zeit der Datenerfassung der Wert unbekannt war
- NULL-Werte führen zu einer dreiwertigen Logik (*wahr, falsch, unbekannt*).
- Diese dreiwertige Logik führt zu:
 - Missverständnissen und
 - bei unvorsichtigem Arbeiten z.B. mit SELECT zu "falschen" Ergebnissen.
- Beispiele sind teilweise dem Artikel "Nulls: Nothing to Worry About" von Lex de Haan und Jonathan Gennick im Oracle Magazine, Juli/August 2005 entnommen

<http://www.oracle.com/technetwork/issue-archive/2005/05-julio45sql-097727.html>, 09.12.2008
DBM-0001-2012-01876

Beispieltabelle DEPT_M



TABLE: DEPT_M

DEPTNO	DNAME	LOC
-----	-----	---
10	HQ	UTRECHT
20	SALES	MUNISING
30	MANUFACTURING	NOVOSIBIRSK

DBM-0001-2012-01877

Beispieltabelle EMP_M



TABLE: EMP_M

EMPNO	ENAME	JOB	MGR	SAL	COMM	DEPTNO
-----	-----	---	---	---	----	-----
100	NORGAARD	PRESIDENT		5000		10
122	LEWIS	SALESREP	120	1100		
199	GENNICK			2200		10
111	DE HAAN	CLERK	110	2000		
112	MILLSAP	SALESREP	110	1250	1400	20
110	ADAMS	MANAGER	100		1700	20
120	KOLK	MANAGER	100	2450		10
113	MCDONALD	SALESREP	110	1500		20
121	WOOD	CLERK	120	1300		10
130	MORLE	CLERK	100			10

DBM-0001-2012-01878

Beispieltabelle EMP_M



TABLE: EMP_M

EMPNO	ENAME	JOB	MGR	SAL	COMM	DEPTNO
----	----	---	---	----	----	-----
100	NORGAARD	PRESIDENT	NULL	5000	NULL	10
122	LEWIS	SALESREP	120	1100	NULL	NULL
199	GENNICK	NULL	NULL	2200	NULL	10
111	DE HAAN	CLERK	110	2000	NULL	NULL
112	MILLSAP	SALESREP	110	1250	1400	20
110	ADAMS	MANAGER	100	NULL	1700	20
120	KOLK	MANAGER	100	2450	NULL	10
113	MCDONALD	SALESREP	110	1500	NULL	20
121	WOOD	CLERK	120	1300	NULL	10
130	MORLE	CLERK	100	NULL	NULL	10

DBM-0001-2012-01879

NULL-Werte und skalare Ausdrücke



- Wie sieht es aus, wenn jeder Angestellte 1000 € mehr bekommt?

```
SELECT      EMPNO, ENAME, SAL, SAL + 1000
FROM EMP_M;
```

- Resultat:

EMPNO	ENAME	SAL	SAL+1000
----	----	---	-----
100	Norgaard	5000	6000
110	Adams	NULL	NULL
111	Dehaan	2000	3000
...			
122	Lewis	1100	2100
130	Morle	NULL	NULL
199	Gennick	2200	3200
...			

DBM-0001-2012-01880

Ergebnis



- Skalare Ausdrücke, die eine NULL umfassen, sind ihrerseits NULL.
- Das führt im Beispiel zuvor dazu, dass Adams und Morle weiterhin ein unbekanntes Gehalt, also NULL erhalten.
- Für den Menschen, der die Bedeutung der Tabelle kennt, ist die Frage, ob es sich bei einem NULL-Gehalt um z.B. folgende Fälle handelt, klar:
 - Gehalt gibt es grundsätzlich nicht für diese Person oder
 - die Person hat ein Gehalt von 0,00 EUR oder
 - die Person hat ein unbekanntes Gehalt, das aber auch um 1000,00 € zu erhöhen ist.
- Der DB-Server kann dies nicht wissen.
- Wenn man also solche Abfragen schreiben will, muss man wissen, welche geschäftliche Bedeutung der NULL-Wert beim Design hatte.

DBM-0001-2012-01881

NULL-Werte und skalare Ausdrücke



- Wenn man z.B. weiß, dass NULL für ein Gehalt von 0,00 € steht, kann man mit der Funktion COALESCE dem NULL-Wert eine Bedeutung geben:

```
SELECT EMPNO, ENAME, SAL, COALESCE(SAL,0) + 1000
      FROM EMP_M;
```

- **Resultat:**

EMPNO	ENAME	SAL	SAL+1000
----	-----	---	-----
100	Norgaard	5000	6000
110	Adams	NULL	1000
111	Dehaan	2000	3000
...			
122	Lewis	1100	2100
130	Morle	NULL	1000
199	Gennick	2200	3200
...			

DBM-0001-2012-01882

SQL-Funktion COALESCE



- **COALESCE** wählt aus seiner Parameterliste (die auch mehr als 2 Argumente haben kann) von links den ersten Wert aus, der nicht NULL ist.

(engl. to coalesce:

vereinigen, zusammenfügen, zusammenwachsen, sich verbinden)

DBM-0001-2012-01883

NULL-Werte und die dreiwertige Logik



- **NULL-Werte** sind insbesondere bei Booleschen Ausdrücken etwa bei
 - WHERE-,
 - HAVING- und
 - ON-Bedingungen
- **tückisch**, da sie einen dritten Wert neben WAHR und FALSCH darstellen:
 - UNBEKANNT.
- Dies führt zu einer dreiwertigen Logik.



DBM-0001-2012-01884

Verknüpfungstabelle der dreiwertigen Logik



x	NOT (x)
-----	-----
FALSE	TRUE
TRUE	FALSE
UNKNOWN	UNKNOWN

▪ **Man beachte, dass NULL != UNKNOWN ist:**

- SAL + NULL ergibt NULL (siehe vorheriges Beispiel) während
- SAL < NULL den Wert UNKNOWN ergibt.

DBM-0001-2012-01885

Verknüpfungstabelle der dreiwertigen Logik



Logisches UND:

x\y	TRUE	FALSE	UNKNOWN
-----+-----+-----+-----			
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

Logisches ODER:

x\y	TRUE	FALSE	UNKNOWN
-----+-----+-----+-----			
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

DBM-0001-2012-01886

Dreiwertige Logik und die WHERE-Restriktion



- Jeder Vergleich mit NULL führt zum Wahrheitswert UNKNOWN, und somit ergibt der folgende SELECT ein auf den ersten Blick seltsames Ergebnis:

```
SELECT ENAME, COMM
      FROM EMP_M
      WHERE COMM = COMM;
```

- Resultat:**

ENAME	COMM
-----	-----
Adams	1700
Millsap	1400

- Die Begründung für das Ergebnis ist, dass die WHERE-Bedingung die Zeilen auswählt, bei denen TRUE herauskommt, die mit FALSE, aber auch die mit UNKNOWN, werden gefiltert!

DBM-0001-2012-01887

Dreiwertige Logik und die WHERE-Restriktion



- Auch der folgende SELECT führt zu einem "seltsamen" Ergebnis:
Alle Angestellten, die weniger als 1500 EUR Kommission (Spalte COMM) erhalten.

```
SELECT ENAME, COMM
      FROM EMP_M
      WHERE COMM < 1500;
```

- Resultat:**

ENAME	COMM
-----	-----
Millsap	1400

- Ist aus Sicht der Anwendung sicher bekannt, dass NULL für "der Angestellte bekommt 0,00 € Provision" steht, kann man mit dem Prädikat IS NULL in der WHERE-Restriktion arbeiten

DBM-0001-2012-01888

Dreiwertige Logik und die WHERE-Restriktion



- Alle Angestellten, die weniger als 1500 EUR Kommission (Spalte COMM) erhalten.

```
SELECT ENAME, COMM
      FROM EMP_M
     WHERE (COMM < 1500) OR (COMM IS NULL);
```

- Resultat:**

```
ENAME      COMM
-----
Norgaard    NULL
Dehaan      NULL
Millsap     1400
...
```

- Warnung:**
Es ist eine anwendungsabhängige Entscheidung, ob NULL für UNBEKANNT oder für 0,00 € steht. Einfach in jeder WHERE-Restriktion noch die NULL-Werte mit IS NULL mitzunehmen, kann auch falsch sein. Die Entscheidung, ob IS NULL angebracht ist oder nicht, ist eine Geschäftsentscheidung, keine technische Entscheidung!

DBM-0001-2012-01889

Dreiwertige Logik und CHECK-Constraints



- WHERE-Restriktionen filtern auch Zeilen heraus, bei denen die Booleschen Verknüpfungen UNKNOWN ergeben,
- Demgegenüber lassen CHECK-Constraints Zeilen mit dem Ergebnis UNKNOWN durch!

Beispiel:
CHECK (DEPTNO IN(10, 20, 30))

- Das liegt daran, dass CHECK-Constraints nur die Zeilen nicht akzeptieren, deren CHECK-Bedingung FALSE ergibt

DBM-0001-2012-01890

OUTER-Joins können NULL erzeugen



- Welcher Mitarbeiter sitzt in welcher Abteilung?

```
SELECT E.ENAME, E.DEPTNO, D.DNAME
      FROM EMP_M E
      RIGHT OUTER JOIN DEPT_M D ON E.DEPTNO = D.DEPTNO;
```

- Resultat:

ENAME	DEPTNO	DNAME
Norgaard	10	HQ
Kolk	10	HQ
...		
Mcdonald	20	Sales
NULL	NULL	Manufacturing
...		

- Hier werden wegen des Schlüsselwortes OUTER automatisch NULL an den Stellen eingefügt, die keinen Partner haben. Interessant ist dabei auch, dass in der letzten Zeile DEPTNO NULL ist, während DNAME nicht NULL ist.

DBM-0001-2012-01891

Null-Werte und Aggregatfunktionen



- Skalare Ausdrücke ergeben den Wert NULL, wenn irgendeine NULL am Ausdruck beteiligt ist.
- Demgegenüber werden NULL-Werte bei Aggregatfunktionen ignoriert!

```
SELECT      SUM(SAL+COMM) AS 'SUM(SAL+COMM) ',
            SUM(SAL)+SUM(COMM) AS 'SUM(SAL)+SUM(COMM) '
      FROM EMP_M;
```

- Resultat:

SUM(SAL+COMM)	SUM(SAL)+SUM(COMM)
2650	19900

DBM-0001-2012-01892

SUM(A) + SUM(B) != SUM(A+B)



Erklärung:

- Hier schlägt die Tatsache zu, dass bei Ausdrücken $\text{NULL} + X = \text{NULL}$ ist, während bei $\text{SUM}(X_1 \dots X_n)$ die NULL-Werte lediglich ignoriert werden.
- Man muss daher, wann immer Ausdrücke oder Aggregatfunktionen einsetzt, sich Gedanken machen, was passiert, wenn ein oder mehrere Werte des selektierten Attributs NULL sind.
- Dazu sollte man vor der eigentlichen Berechnung mit `SELECT ... WHERE ... IS NULL` prüfen, ob NULL-Werte mitspielen.
- Auch in diesem Beispiel kann man nicht eindeutig sagen, welcher Ansatz der richtige ist. Das hängt wieder von der Bedeutung der NULL ab. Die kann sogar von Spalte zu Spalte verschieden sein!

DBM-0001-2012-01893

Was ist die Summe von 0 Zeilen?



Fast schon philosophisch ist folgender SELECT

```
SELECT      COUNT (EMPNO) , AVG (EMPNO) , SUM (EMPNO) ,
            MAX (EMPNO) , MIN (EMPNO)
FROM EMP_M
WHERE 1 = 2;
```

Resultat:

COUNT (EMPNO)	AVG (EMPNO)	SUM (EMPNO)	MAX (EMPNO)	MIN (EMPNO)
0	NULL	NULL	NULL	NULL

Erklärung:

- Dass der COUNT von 0 Zeilen 0 ergibt, ist klar.
- Dass es bei 0 Zeilen nur ein UNBEKANNTes Maximum und Minimum gibt, ist klar.
- Aber dass die Summe von 0 Zeilen UNBEKANNT und nicht 0 ist, ist zwar nicht klar, steht aber so im SQL-Standard.

DBM-0001-2012-01894

Null-Werte und Unterabfragen



- Auch bei Unterabfragen sind NULL-Werte tückisch, wenn z.B. die Unterabfrage auch einen NULL-Wert erzeugt.
- dann kann ein NOT IN leicht zum Ergebnis UNBEKANNT führen, was wiederum dazu führt, dass gar keine Zeile der Hauptabfrage die WHERE-Restriktion passieren kann, wie im folgenden Beispiel gezeigt.

DBM-0001-2012-01895

Null-Werte und NOT IN I/II



- Alle Angestellten, die keine Untergebenen haben

```
SELECT E1.ENAME
      FROM EMP_M E1
      WHERE E1.EMPNO NOT IN
            (SELECT E2.MGR
              FROM EMP_M E2);
```

- **Resultat:**

```
ENAME
-----
(0 row(s) affected)
```

DBM-0001-2012-01896

Null-Werte und NOT IN II/II



- Alle Angestellten, die keine Untergebenen haben:

```
SELECT E1.ENAME
      FROM EMP_M E1
      WHERE E1.EMPNO NOT IN
            (SELECT E2.MGR
              FROM EMP_M E2
              WHERE E2.MGR IS NOT NULL);
```

- Resultat:

```
ENAME
-----
Dehaan
Millsap
...
Morle
Gennick
(7 row(s) affected)
```

DBM-0001-2012-01897

Unterabfrage mit EXISTS



- Alle Angestellten, die keine Untergebenen haben

```
SELECT E1.ENAME
      FROM EMP_M E1
      WHERE NOT EXISTS
            (SELECT *
              FROM EMP_M E2
              WHERE E2.MGR = E1.EMPNO);
```

- Resultat:

```
ENAME
-----
Dehaan
Millsap
...
Morle
Gennick
(7 row(s) affected)
```

DBM-0001-2012-01898

Unterabfragen, die die leere Menge liefern



- Alle Angestellten, die mehr als irgendein Verkäufer (engl. Salesrep) in Abteilung 10 verdienen:

```
SELECT E1.ENAME
FROM EMP_M E1
WHERE E1.SAL >
      (SELECT MAX(E2.SAL)
       FROM EMP_M E2
        WHERE E2.DEPTNO = 10
              AND E2.JOB='SALESREP');
```

- Resultat:

```
ENAME
-----
(0 row(s) affected)
```

DBM-0001-2012-01899

Unterabfragen, die die leere Menge liefern



- Alle Angestellten, die mehr als irgendein Verkäufer in Abteilung 10 verdienen:

```
SELECT E1.ENAME
FROM EMP_M E1
WHERE E1.SAL > ALL
      (SELECT E2.SAL
       FROM EMP_M E2
        WHERE E2.DEPTNO = 10
              AND E2.JOB = 'SALESREP');
```

- Resultat:

```
ENAME
-----
Norgaard
Adams
...
Gennick
(10 row(s) affected)
```

DBM-0001-2012-01900